1. Nazwa klas i metod

1.1 **Nazwy klas**: Używaj notacji CamelCase (np. Chess, Pawn, King).
1.2 **Nazwy metod**: Używaj notacji snake_case (np. handle_square_click, move_piece).

---

2. Komentarze

2.1 **Komentowanie kodu**: Komentuj kod w sposób zrozumiały i klarowny, wyjaśniając, co robi każda funkcja, metoda lub fragment kodu.
2.2 **Język komentarzy**: Używaj języka angielskiego w komentarzach, aby kod był zrozumiały dla szerszej grupy programistów.
2.3 **Dokumentowanie złożonych funkcji**: Zawsze dodawaj komentarze wyjaśniające logikę szczególnie złożonych funkcji i metod.

---

3. Formatowanie kodu

3.1 **Wcięcia**: Używaj tabulatorów do wcięć w kodzie.
3.2 **Puste linie**: Stosuj puste linie między metodami i klasami, aby poprawić czytelność kodu i oddzielić różne sekcje w kodzie.

---

4. Struktura kodu

4.1 **Podział na mniejsze fragmenty**: Jeśli dana funkcja lub metoda staje się zbyt długa, rozważ podzielenie jej na mniejsze, bardziej zrozumiałe fragmenty.
4.2 **Zasada pojedynczej odpowiedzialności**: Każda funkcja/metoda powinna odpowiadać tylko za jedną konkretną rzecz, co ułatwia jej zrozumienie i testowanie.

---

5. Nazwy zmiennych

5.1 **Jasne nazwy**: Używaj nazw zmiennych, które jednoznacznie opisują ich przeznaczenie. Zmienna powinna mieć pełny opis, np. start_pos zamiast pos.

5.2 **Unikanie skrótów**: Unikaj skrótów, które mogą być trudne do zrozumienia w kontekście kodu. Na przykład piece_color jest lepsze niż p_col.

---

6. Unikanie magicznych liczb

6.1 **Stałe zamiast magicznych liczb**: Unikaj tzw. magicznych liczb (np. 8, 2, 3), które mogą być trudne do zrozumienia.
6.2 **Zamiast magicznych liczb wprowadź stałe**: Zamiast magicznych liczb, wprowadź stałe z odpowiednimi nazwami, np. BOARD_SIZE = 8, MAX_PIECES = 16.

---

7. Zarządzanie błędami

7.1 **Obsługa błędów**: Dodaj odpowiednie mechanizmy obsługi błędów i wyjątków tam, gdzie może dojść do problemów, np. podczas wykonywania ruchów, sprawdzania szacha czy roszady.
7.2 **Sprawdzanie poprawności ruchów**: Zawsze sprawdzaj, czy ruchy są legalne i czy nie prowadzą do błędów, np. ruchy, które łamią zasady gry.

---

8. Spójność kodu

8.1 **Jednolity styl**: Używaj jednolitych struktur i stylu w całym projekcie, np. nazewnictwa zmiennych, formatowania kodu i stylu komentarzy.
8.2 **Ułatwienia dla współpracowników**: Spójny styl kodu ułatwia zrozumienie i edytowanie kodu przez innych programistów w przyszłości.

Przykładowy fragment kodu:

```
class Chess:
    def __init__(self):
        self.board_row = 8
        self.board_col = 8
        self.turn = 'white'
        self.grid = None
        self.board = self.initialize_board()
        self.squares = [[None for _ in range(self.board_col)] for _ in range(self.board_row)
        self.white_captured = []
        self.black_captured = []
        self.last_move = None
        self.create_chessboard()

    def initialize_board(self):
        """Inicjalizuje planszę szachową na początkową konfigurację figur."""
        return [
            [Rook('white', 0, 0), Knight('white', 0, 1), Bishop('white', 0, 2), Queen('white
             King('white', 0, 4), Bishop('white', 0, 5), Knight('white', 0, 6), Rook('white'
            [Pawn('white', 1, i, self) for i in range(8)],
            [None] * 8,
            [None] * 8,
            [None] * 8,
            [None] * 8,
            [Pawn('black', 6, i, self) for i in range(8)],
            [Rook('black', 7, 0), Knight('black', 7, 1), Bishop('black', 7, 2), Queen('black
             King('black', 7, 4), Bishop('black', 7, 5), Knight('black', 7, 6), Rook('black'
        ]

    def handle_square_click(self, i, j):
        """Obsługuje kliknięcie na danym polu szachownicy."""
        piece = self.board[i][j]
        if self.start_pos is None:
            if piece and self.turn == piece.color:
                self.start_pos = (i, j)
                ui.notify(f'Wybrano figurę na ({i}, {j})', color='blue')
            else:
                ui.notify('Nie możesz ruszyć tej figury!', color='red')
        else:
            start_row, start_col = self.start_pos
            piece = self.board[start_row][start_col]

            if piece and (i, j) in piece.available_moves(self.board, self.last_move):
                self.move_piece(start_row, start_col, i, j)
            else:
                ui.notify('Nieprawidłowy ruch', color='red')
            self.start_pos = None
```